Advanced Modular Telegram Bot

Architecture & Code Reference

1. LAYERED ARCHITECTURE

The bot follows a strict layered architecture where each layer has a single responsibility and dependencies point INWARD (main -> handlers -> services -> db).

```
  advanced_bot_reference/
  +-- main.py                    Entry point - wires everything together
  +-- config.py                  Frozen dataclass config from env vars
  +-- requirements.txt           Dependencies (aiogram, aiosqlite, fpdf2, dotenv)
  +-- bot/
  |   +-- loader.py              Bot & Dispatcher singleton
  |   +-- middlewares.py         Middleware pipeline (4 middlewares)
  +-- database/
  |   +-- models.py              Dataclass models (User, Group, Broadcast, Stats)
  |   +-- core.py                Async SQLite operations + singleton
  +-- filters/
  |   +-- admin.py               Composable filters (Admin, Premium, NotBanned)
  +-- keyboards/
  |   +-- common.py              Keyboard factory + pre-built keyboards
  +-- services/
  |   +-- broadcast.py           Robust mass messaging with retry/ban-detection
  |   +-- stats.py               Query + format system statistics
  +-- handlers/
  |   +-- user.py                User commands + FSM workflows
  |   +-- admin.py               Admin panel + broadcast FSM
  |   +-- group.py               Group join/leave lifecycle
  |   +-- errors.py              Global error handler + unhandled fallback
  +-- data/
      +-- bot.db                 SQLite database (auto-created)
```

```
Data flow:  main.py -> handlers -> services -> database/core.py -> SQLite
Middleware flow:  Throttling -> UserRegistration -> CommandLogging / ChatAction
```

2. CONFIG (config.py)

Uses a frozen dataclass with slots=True for memory efficiency and immutability. The from_env() classmethod validates that BOT_TOKEN is present and parses ADMIN_IDS from a comma-separated env variable. The is_webhook property drives whether the bot starts in polling or webhook mode.

```
  @dataclass(frozen=True, slots=True)
  class Config:
      BOT_TOKEN: str                  # required - raises ConfigError if missing
      ADMIN_IDS: List[int]            # parsed from "123,456" env var
      BOT_MODE: str = "polling"       # polling | webhook
      WEBHOOK_URL: str = ""
      WEBHOOK_PORT: int = 8443
      DB_PATH: Path = "data/bot.db"
      LOG_LEVEL: str = "INFO"
      THROTTLE_RATE: float = 0.5      # seconds between allowed user actions
```

```
  cfg = Config.from_env()  # module-level singleton
```

3. DATABASE LAYER

```
3a. Models (database/models.py) - Dataclasses

  UserModel:      id, telegram_id, username, first_name, last_name,
                  language, is_premium, is_banned, is_admin,
                  total_commands, last_activity, created_at
  GroupModel:     id, telegram_id, title, username, member_count,
                  is_active, welcome_enabled, welcome_message, created_at
  BroadcastModel: id, admin_id, message_text, sent_count,
                  total_count, failed_count, status, created_at, completed_at
  StatsModel:     total_users, total_groups, total_commands,
                  active_today, new_today

3b. Core (database/core.py) - Async SQLite Operations

  KEY PATTERNS:
  * @asynccontextmanager conn(): opens/closes per call (safe, simple)
  * Upsert: INSERT ... ON CONFLICT(telegram_id) DO UPDATE SET ...
  * Whitelist updates: update_user() only accepts known field names
  * WAL mode: PRAGMA journal_mode=WAL for concurrent reads
  * Singleton: get_db() caches Database instance globally

  CORE METHODS:
  register_user(telegram_id, **kwargs) -> UserModel
      INSERT OR UPDATE user with current profile data

  get_user(telegram_id) -> UserModel | None
  update_user(telegram_id, **kwargs)
      Only allows: username, first_name, last_name, language,
      is_premium, is_banned, is_admin, total_commands, last_activity

  increment_commands(telegram_id)
      Increments total_commands and updates last_activity

  get_all_users(only_active=False) -> List[UserModel]
  get_user_count() -> int
  get_active_today_count() -> int
  get_new_today_count() -> int

  register_group / get_group / update_group / get_all_groups / get_group_count

  create_broadcast(admin_id, text, total) -> broadcast_id
  update_broadcast(broadcast_id, **kwargs)
  get_broadcasts(limit=10) -> List[BroadcastModel]

  log_command(telegram_id, command, args)
  get_top_commands(limit=10) -> List[Dict]
  get_command_count() -> int

  get_stats() -> StatsModel
```

4. MIDDLEWARE PIPELINE (bot/middlewares.py)

Four middlewares registered in order. Each has a single responsibility.

1. THROTTLING MIDDLEWARE
   Drops events from the same user if they arrive faster than THROTTLE_RATE.
   Uses a dict mapping user_id -> last_event_time.

   ```
   class ThrottlingMiddleware(BaseMiddleware):
       def __init__(self, rate: float = 0.5):
           self.rate = rate
           self.users: Dict[int, float] = {}
       # Returns None (dropped) if interval < rate
   ```

2. USER REGISTRATION MIDDLEWARE
   On every message/callback, upserts the user into the DB.
   Ensures all users are registered without manual /start handling.

   ```
   class UserRegistrationMiddleware(BaseMiddleware):
       async def __call__(self, handler, event, data):
           user = data.get("event_from_user")
           if user and not user.is_bot:
               db = get_db()
               await db.register_user(id=user.id, username=user.username, ...)
           return await handler(event, data)
   ```

3. COMMAND LOGGING MIDDLEWARE
   After the handler runs, checks if the message is a /command.
   Logs to the logs table and increments total_commands.

   ```
   class CommandLoggingMiddleware(BaseMiddleware):
       async def __call__(self, handler, event, data):
           result = await handler(event, data)
           if isinstance(event, Message) and event.text.startswith("/"):
               # log command + increment
           return result
   ```

4. CHAT ACTION MIDDLEWARE
   Auto-answers callback queries so every button press stops the loading spinner.

   ```
   class ChatActionMiddleware(BaseMiddleware):
       async def __call__(self, handler, event, data):
           if isinstance(event, CallbackQuery):
               await event.answer()
           return await handler(event, data)
   ```

5. FILTERS (filters/admin.py)

Composable access control. Attached at router level to batch-apply protection.

AdminFilter:
    Checks if user.id is in cfg.ADMIN_IDS (env-defined super admins)
    OR if the DB record has is_admin=True.

PremiumFilter:
    Checks if the DB user has is_premium=True.

NotBannedFilter:
    Blocks banned users entirely. Applied to user_router so banned users
    can't call any commands (except cannot be applied to errors router).

```
  USAGE:
      user_router.message.filter(NotBannedFilter())
      admin_router.message.filter(AdminFilter())
      admin_router.callback_query.filter(AdminFilter())
```

6. KEYBOARDS (keyboards/common.py)

```
KeyboardBuilder is a static factory class with 4 methods:

  KeyboardBuilder.inline(buttons: List[List[Tuple[str, str]]])
      -> InlineKeyboardMarkup
      Builds inline buttons from (text, callback_data) pairs.

  KeyboardBuilder.url_inline(buttons: List[List[Tuple[str, str]]])
      -> InlineKeyboardMarkup
      Builds inline buttons from (text, url) pairs.

  KeyboardBuilder.reply(buttons: List[List[str]])
      -> ReplyKeyboardMarkup
      Builds reply keyboard from plain text labels.

  KeyboardBuilder.remove()
      -> ReplyKeyboardRemove
      Removes the current keyboard.

  PRE-BUILT KEYBOARDS:
  main_menu_kb()   -> Profile | Stats, Settings | Help, About
  admin_menu_kb()  -> Dashboard | Broadcast, Users | Groups, Logs | Back
  cancel_kb()      -> [Cancel] reply button
  confirm_kb()     -> Yes | No inline
  settings_kb()    -> Language, Back
```

7. SERVICES LAYER

```
7a. BroadcastService (services/broadcast.py)

  Robust mass messaging with error handling:

  send_to_user(user_id, text, keyboard) -> bool
      Handles: TelegramForbiddenError (auto-bans the user)
               TelegramRetryAfter (sleeps and retries)
               TelegramBadRequest (skips)

  broadcast(text, admin_id, target_ids=None) -> int
      Creates a broadcast record in the broadcasts table.
      Iterates all non-banned users (or custom target_ids list).
      Updates progress every 10 sends.
      iddle 50ms between sends to avoid flood limits.
      Supports cancellation via self.cancel().
      Returns count of successful sends.

  cancel() -> None
      Sets _is_running = False, which stops the broadcast loop.

7b. StatsService (services/stats.py)
```

```
  get_stats_text() -> str
      Queries: total users, active today, new today,
      total groups, total commands, top 5 commands.
      Returns formatted string for display.
```

8. HANDLERS

```
8a. User Handler (handlers/user.py)

  Commands:
    /start   - Welcome + main menu (main_menu_kb)
    /help    - List all available commands
    /profile - Show user's telegram_id, username, language, premium status, commands
    /stats   - Show system-wide statistics
    /settings - Show settings menu (language picker)
    /feedback - FSM: collect text -> forward to all admins
    /about   - Architecture summary
    /cancel  - Clear any active FSM state

  FSM Workflows:
    SettingsForm:
      set_language_start -> shows reply keyboard with language options
      set_language_done  -> validates, updates DB, clears state

    FeedbackForm:
      feedback_start     -> sets state, shows cancel_kb
      feedback_done      -> forwards text to all ADMIN_IDS
      feedback_invalid   -> catches non-text input

  Callback handlers mirror commands for inline navigation.

8b. Admin Handler (handlers/admin.py)

  Protected by AdminFilter on the router.

  Commands:
    /admin       - Show admin panel menu
    /ban <id>    - Set is_banned=True
    /unban <id>  - Set is_banned=False
    /premium <id> - Toggle is_premium

  FSM Workflow:
    BroadcastForm:
      admin_broadcast_start   -> enters FSM, asks for text
      admin_broadcast_preview -> shows preview + confirm_kb
      admin_broadcast_execute -> runs BroadcastService.broadcast()
      admin_broadcast_cancel  -> clears state

  Callback panels:
    admin_dashboard -> system stats
    admin_users     -> user counts (total, banned, admins, premium)
    admin_groups    -> list active groups
    admin_logs      -> top 10 commands

8c. Group Handler (handlers/group.py)

  Restricted to group/supergroup chats via F.chat.type filter.
```

```
  on /start in group:
      Registers the group in the groups table.

  on my_chat_member(KICKED):
      Sets is_active = False for the group.

  on my_chat_member(MEMBER):
      Registers/syncs the group.

8d. Error Handler (handlers/errors.py)

  global_error_handler:
      1. Logs full traceback
      2. Notifies the affected user
      3. Notifies all admins with error type, detail, and user_id

  unhandled_message:
      Catches unknown commands -> responds with "Unknown command.
```

9. ENTRY POINT (main.py)

```
  setup_logging() -> configures logging to stdout
  setup_bot_commands() -> sets 8 commands in Telegram's / menu
  register_middlewares() -> attaches all 4 middlewares to dispatcher
  register_all_routers(dp) -> includes user, admin, group, error routers

  on_startup():
      1. Runs database setup (creates tables + indexes)
      2. Registers bot commands
      3. Registers middlewares
      4. Registers all routers

  main():
      1. Setup logging
      2. Register startup/shutdown hooks
      3. Delete webhook (drop pending) if SKIP_UPDATES=True
      4. Start polling OR webhook based on config
```

10. KEY DESIGN DECISIONS

DECISION	WHY
SQLite (no Redis)	Zero external deps - just BOT_TOKEN needed
Frozen dataclass config	Immutable; can't be mutated at runtime
Async contextmanager per query	Simpler than pooling; safe with SQLite WAL
Whitelist on update_user()	Can't accidentally set arbitrary fields
Filters on routers, not handler	Batch-apply; cleaner handler code
Static KeyboardBuilder factory	DRY keyboard creation; centralized
Broadcast as service class	Cancellable, progress-tracked, error-resilient

11. FLOW EXAMPLE: /start

```
  1. Bot receives /start message
  2. ThrottlingMiddleware checks user isn't spamming
  3. UserRegistrationMiddleware upserts user into DB
  4. cmd_start() handler runs -> sends welcome + menu inline keyboard
  5. CommandLoggingMiddleware logs "start" to logs table
```

6. total_commands incremented for the user

12. TO RUN

```
cp .env.example .env
# Edit .env: set BOT_TOKEN (and ADMIN_IDS if needed)
pip install -r requirements.txt
python main.py

Bot will start in polling mode by default.
Set BOT_MODE=webhook and WEBHOOK_URL for production deployment.
```